



Politehnica University of Timisoara



Web Programming

C8. AI-driven Web development and applications



AI-Driven Web Development

- Artificial intelligence is reshaping web development across the entire stack
 - UI design
 - Front-end code generation
 - Back-end scaffolding
 - Data management and deployment
 - Testing
 - Observability and performance tuning
 - Security and accessibility auditing
 - New development tools





Design and prototyping

- Generative Models and specialized Vision-to-Code Systems convert hand sketches and mockups into HTML/CSS
- Reduces the manual translation step between UI design and front-end implementation
- Benefits - faster prototyping, alignment between designer intent and live UI, and improved design consistency when paired with component libraries
- Limitations - generated layouts often need refinement for responsiveness, edge cases, and production accessibility



Code generation and assistance

- Tools powered by code-trained LLMs provide context-aware autocompletion, whole-function synthesis, refactor suggestions, and even architectural scaffolding (routes, data models, CRUD APIs)
- Increase developer productivity for repetitive code
- Benefits - fewer boilerplate errors, quicker onboarding, and higher throughput for routine tasks
- Limitations - hallucinated or insecure code, license concerns, and a need for human review



Low-code platforms

- Platforms integrate AI to let developers build web applications via conversational interfaces, which allow adding functional requirements in natural language
- They can generate front-end and back-end code and wiring, database schemas, and basic business logic code
- Benefits - empowers non-technical users, reduces time-to-market for prototypes
- Limitations - limited flexibility for complex business rules, potential vendor lock-in, and maintainability issues as products scale up



Testing and QA

- AI assists can explain code, generate test cases, detect regressions via anomaly detection, and prioritize bugs
- For runtime systems, ML models can flag unusual latency patterns or likely root causes for incidents, improving mean-time-to-repair
- Benefits - broader automated test coverage, faster debugging, improved reliability
- Limitations - false positives/negatives and poor model explainability that complicate trust



Performance and UI personalization

- AI models help optimize resource loading (e.g., recommending critical CSS for lazy-loading strategies), predictively fetch resources, or personalize content and UI flows based on user behavior
- Benefits - improved perceived performance and conversion through targeted optimizations
- Limitations - data privacy concerns, model drift (as users and content change), and the risk of creating opaque personalization that frustrates users



Risks, limitations, ethical considerations

- Code correctness and security - generated code can contain logic bugs or insecure patterns, resulting in additional costs for continuous code review and testing
- Intellectual property - models trained on public repositories raise licensing questions
- Bias and accessibility - automated UI generation may perpetuate accessibility problems if checks are not applied
- Maintainability - AI-generated code that does not follow the team's conventions may increase costs
- Privacy - personalization must comply with data-protection regulations and transparent consent practices



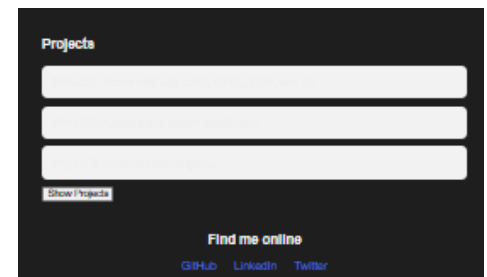
Tools: ChatGPT

- Use prompts like: “Create a simple personal homepage for me using HTML, CSS, and JavaScript. It should include a profile photo, a short bio, links to my social media, and a light/dark mode toggle.”
- A detailed description and guidance are necessary to reduce hallucination
- Additional (private) resources should be added (e.g., the profile photo)
- Prompt can include requests to use a specific library (e.g., jQuery) or framework (e.g., React or Express)



Prompts for ChatGPT

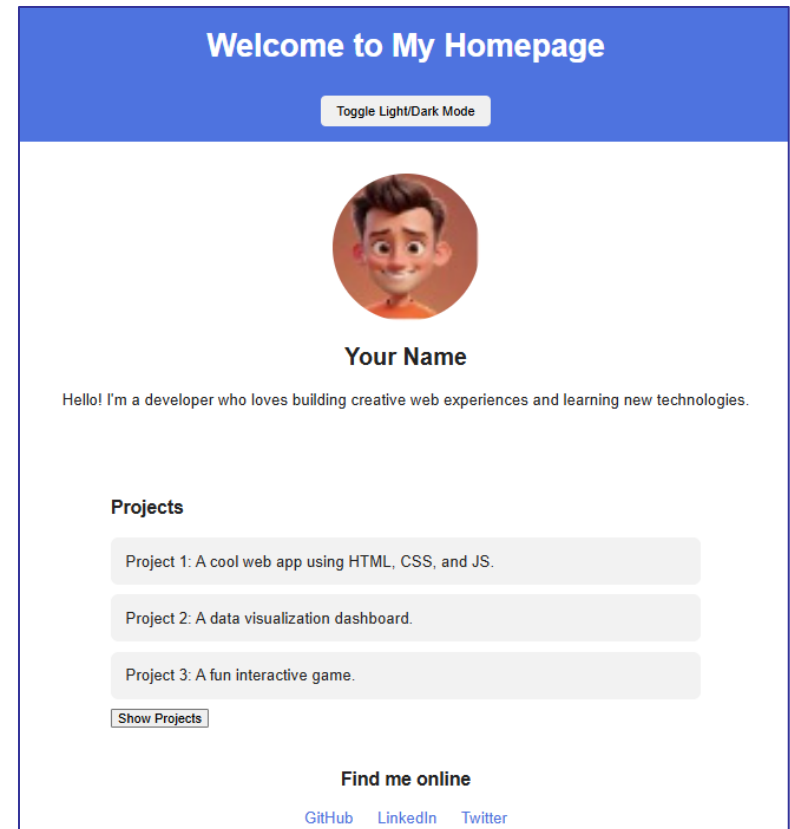
- More details will guide the generation to specific requirements like
- “Create a personal homepage for me using HTML, CSS, JavaScript, and jQuery. It should include a profile photo, a short bio, a list of projects animated in jQuery, and links to my social media, and a light/dark mode toggle.”
- However, this can generate UI issues, as in the example (e.g., white on white Projects text for dark mode)





Refining the Prompt for ChatGPT

- Request for correction can be further specified, like
- “Modify the project to display the projects list's text in black when displayed on dark mode.”





Use ChatGPT for SEO

- Use prompts like: “Suggest optimizations for the project to improve search rankings in search engines.”
- The answer was:

I can update the canvas with:

- semantic HTML
- SEO meta tags
- updated alt texts
- structured data
- improved layout and heading structure

Just tell me “Yes, update my homepage with these SEO changes.”



SEO Improvements Summary

- Meta Tags Added
 - Description, keywords, and author meta tags
 - Open Graph tags (improves link previews on Facebook/LinkedIn)
 - Twitter Card tags (improves sharing on Twitter/X)
- Structured Data (Schema.org)
 - Inserted JSON-LD "Person" schema with name, image, social links, and job title to help Google display richer results
- Improved Semantic HTML
 - Wrapped main content in `<main>` to improve structure and crawlability to help search engines understand the hierarchy of the page
- Accessibility Upgrade
 - Updated profile image alt text to a descriptive form: "Portrait of Your Name, web developer"
- Improved Head Section structure
 - Meta tags and data now appear correctly before `<style>`



Code support tools

■ GitHub Copilot



- Free for students and open-source projects
- Tight integration with GitHub, cloud-based processing
- Boost rapid development, powerful code generation
- Generate blocks and functions in Python, JS, and C++

■ Tabnine



- Best for enterprise environments
- Customizable to organization-specific coding style and best practices
- Priority is privacy, on-premise deployment, and team customization (user code is not used for training)



GitHub Copilot in VS Code

The screenshot shows the VS Code interface with a code editor on the left and a chat sidebar on the right. The code editor displays the following TypeScript code:

```
1 import { Todo } from '../types/todo.types';
2 import { StorageService } from './StorageService';
3
4 export class ChatTodoListService {
5   private storageService: StorageService;
6   private eventListeners: ((todos: Todo[]) => void)[] = [];
7
8   constructor() {
9     this.storageService = new StorageService();
10  }
11
12  // Existing methods (read/write)
13  getTodos(): Todo[] {
14    return this.storageService.get('todos') || [];
15  }
16
17  setTodos(todos: Todo[]): void {
18    this.storageService.set('todos', todos);
19    this.emitChange(todos);
20  }
21
22  // NEW METHOD - Step 1 of the plan
23  updateTodo(updatedTodo: Todo): void {
24    const todos = this.getTodos();
25    const index = todos.findIndex(todo => todo.id === updatedTodo.id);
26
27    if (index !== -1) {
28      todos[index] = { ...todos[index], ...updatedTodo };
29      this.storageService.set('todos', todos);
30      this.emitChange(todos);
31    }
32  }
33 }
```

The chat sidebar on the right contains the following content:

CHAT

allow editing the todo title in the chat tool list

Add interactive checkbox toggling and inline editing to the chat todo list widget for manual task updates alongside AI management.

Steps:

1. Implement updateTodo() in ChatTodoListService for individual updates
2. Add click handlers to cycle status: not-started → in-progress → completed
3. Add inline title editing via double-click with save/cancel actions
4. Extend ManageTodoListTool with update operation for AI modifications
5. Update tests for interactions, status toggling, and editing flows

Open Questions:

1. Conflict resolution? (AI overwrites / User locks / Merge with timestamps)
2. User add/delete todos? (UI buttons / AI-only / Context menu)
3. Description editing? (Inline / Modal / Title-only)

PROCEED FROM PLAN

Start Implementation Open Plan in Editor

@agents, /comments

Plan Auto

Ref: <https://github.com/features/copilot/ai-code-editor>

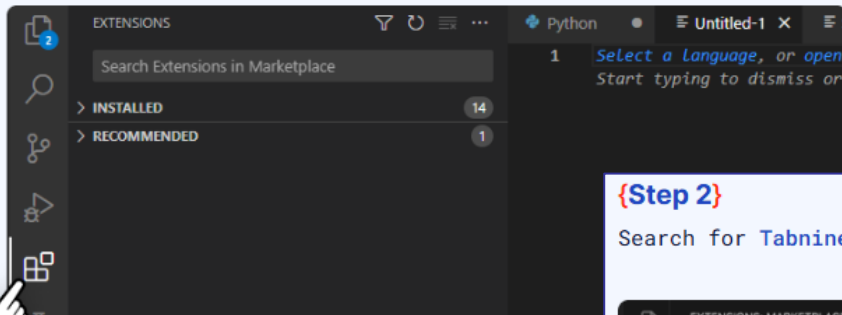


Tabnine in VS Code

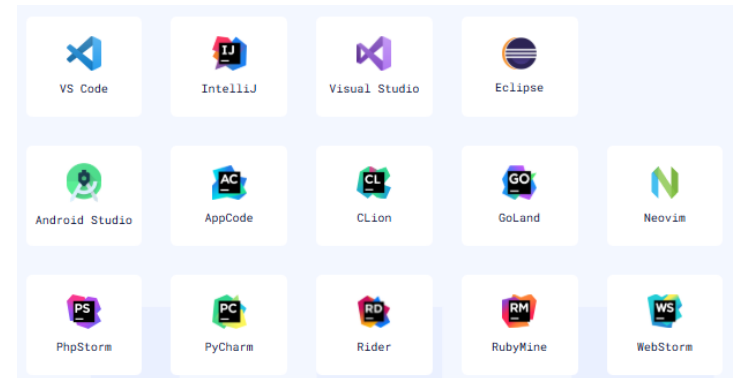
■ In Visual Studio Code

{Step 1}

In the left sidebar, click the **Extensions** icon.

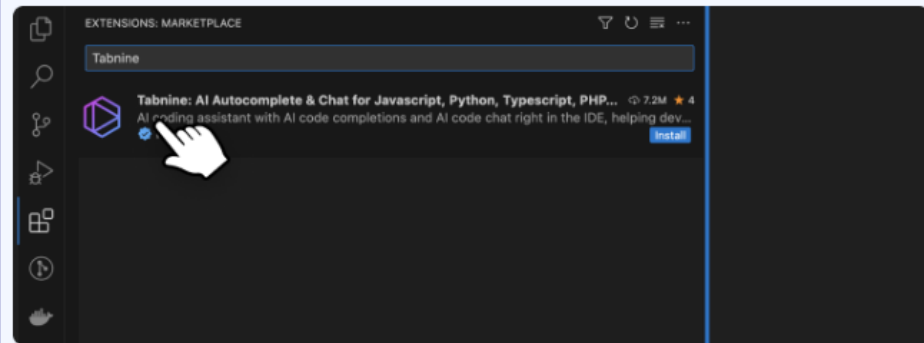


Ref: <https://www.tabnine.com/install/vs-code/>



{Step 2}

Search for **Tabnine** and click **Install**. Restart VS Code.





AI for Web apps testing

- DeepScan (deepscan.io)

- Outstanding in inspecting JavaScript code quality
- Evaluate the code quality status of team projects
- Get quality metrics on the project



- Codacy (www.codacy.com/)

- Application Security for AI-Accelerated Coding
- Enforces security and quality standards
- IDE Integration, AI Code Analysis, Security attack simulation



- Testim (www.testim.io)

- AI-powered agentic test automation
- AI to create tests autonomously using natural language
- Designed for Web and Mobile applications





AI empowered Web apps

- Chatbots and AI-powered virtual assistants
 - Provide instant support and assistance to users
 - Improve customer service and user engagement
 - AI chatbots – HubSpot (www.hubspot.com/products/crm), Tidio (tidio.com), and Drift (salesloft.com/platform/drift)
- AI-powered chatbots for
 - Answering FAQs
 - Guiding users through processes
 - Helping with troubleshooting
- Virtual assistants can perform tasks like
 - Scheduling appointments
 - Providing personalized recommendations
 - Initiating transactions



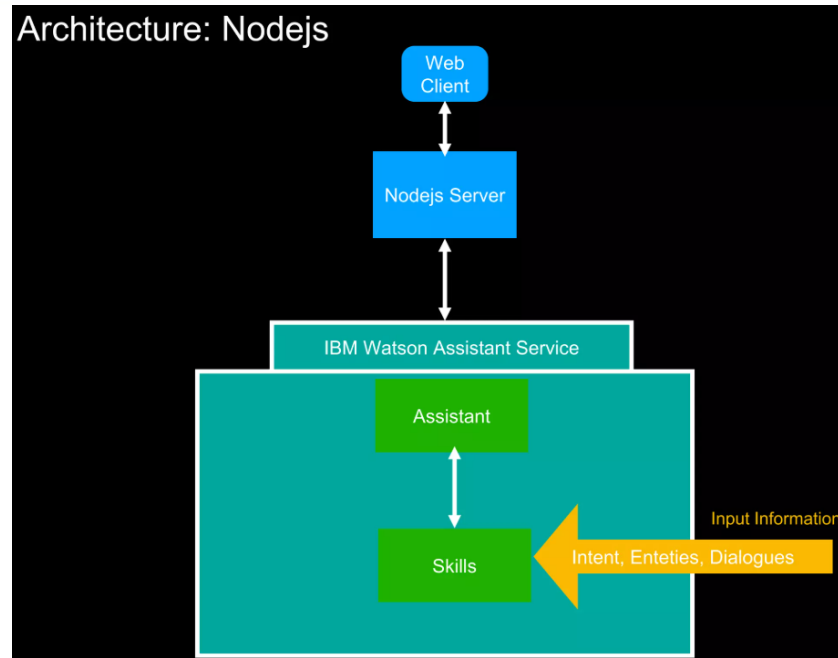
IBM Watson Assistant

- AI Assistant for business applications, designed to
 - Answer questions
 - Help users to complete tasks across any application, device, or channel
- Allows building conversational agents or "skills" that can
 - Understand natural language conversations
 - Search for answers in a company knowledge base
 - Ask for clarifications
 - Decide to request human help, if necessary
 - Run a task with customized parameters extracted from the conversation





IBM Watson – Web Integration



- Skill – a software component for solving a task (e.g., weather forecast provider)
- Intent – service that users need from a Skill (e.g., #temperature_for_tonight)
- Entity – objects that can provide context for an intent in conversation (e.g., city name)
- Dialog – the conversation flow between a user and a skill



IBM Watson Assistant implementation

Three easy steps

Follow these steps to create an assistant.

1

Build a skill

A skill contains the training data and machine learning logic that enable your chat bot to understand and help customers. Add training data built by IBM or use graphical tools that help you build your own.

[Learn more](#)

2

Deploy with an assistant

An assistant orchestrates the flow of information between a skill and your customers. Add your skill to an assistant, and then deploy the assistant to the media channel of your choice.

[Learn more](#)

3

Analyze and improve

Use metrics collected from conversations with real customers to iteratively improve your skill.

[Learn more](#)

Get Started Now

Create a Skill

Ref: www.ibm.com/products/watsonx-assistant

Banking assistant

Transfer money

Which account do you want to transfer from?

Actually what's my balance in my savings?

Your Savings balance: \$5,213.00

Which account do you want to transfer from?

Savings into checking



Deploy ML models using Flask



- Flask is a lightweight Web Framework for Python
- Flask's routing system maps URLs to Python functions

```
import flask
import pickle

app=flask.Flask(__name__) # creating instance of the class
@app.route('/')           # what url will trigger the function index()
@app.route('/index')

def index():
    return flask.render_template('index.html')

def ValuePredictor(to_predict_list):
    to_predict = np.array(to_predict_list).reshape(1,12)
    loaded_model = pickle.load(open(r"path_to_the_saved_model", "rb"))
    result = loaded_model.predict(to_predict)
    return result[0]

@app.route('/result', methods = ['POST'])
def result():
    if request.method == 'POST':
        to_predict_list = request.form.to_dict()
        to_predict_list=list(to_predict_list.values())
        to_predict_list = list(map(int, to_predict_list))
        result = ValuePredictor(to_predict_list)

        if int(result)==1:
            prediction='Classified as Class_1'
        else:
            prediction='Classified as Class_2'
        return render_template("result.html", prediction=prediction)

if __name__ == "__main__":
    app.run(debug=True)
```



References

- OpenAI. Introducing Codex - Cloud-based software engineering agent, 2021
<https://openai.com/index/introducing-codex/>
- GitHub. GitHub Copilot - Your AI pair programmer
<https://github.com/features/copilot>
- Microsoft. CodeBERT - Pre-Trained Model for Programming and Natural Languages, 2020
<https://github.com/microsoft/CodeBERT>
- IBM Watson & Watsonx - portfolio of AI products
<https://www.ibm.com/watson>